Distributed File Retrieval System Architecture

1. System Overview

The implemented system is a **simple distributed file retrieval system** consisting of **one client and two file servers**. The two servers maintain replicas of the same set of files. Since updates may not propagate immediately, the replicas may temporarily diverge. The system therefore performs **consistency validation before returning files to the client**.

The architecture follows a **coordinator-based distributed model**, where one server acts as the primary coordinator responsible for managing client requests and verifying replica consistency.

---

2. System Components

```
CLIENT —---> SERVER1 <------> SERVER2
             (Coordinator)    (Replica)
```

Client

The **CLIENT** is responsible for initiating file requests. It sends a file pathname to SERVER1 and waits for a response. Upon receiving the response, the client decodes the file content and stores it locally.

Server1 (Coordinator)

SERVER1 acts as the primary server and coordinator. Its responsibilities include:

- Receiving file requests from the client.
- Checking its own local storage for the requested file.
- Sending a metadata request to SERVER2 to verify replica status.
- Comparing file hashes to determine if replicas are consistent.
- Returning the correct response to the client.

SERVER1 implements the decision logic that determines how the system handles different file availability scenarios.

Server2 (Replica)

SERVER2 acts as a replica file server. It maintains a copy of the file set and responds to metadata requests from SERVER1. Its responsibilities include:

- Checking whether the requested file exists.
- Computing the SHA-256 hash of the file.
- Returning metadata (existence, hash value, file size) to SERVER1.

SERVER2 does not communicate directly with the client.

The client communicates only with SERVER1, while SERVER1 communicates with SERVER2 to verify file consistency.

In short:

Client → Sends file requests

Server1 → Coordinator server handling requests

Server2 → Replica file server

---

3. Phase 1 – System Model Definition

3.1 Objective → The objective of this phase was to design the overall distributed system architecture, define communication paths, and identify the roles of each node in the system.

Theory and Concepts: This phase demonstrates the following distributed system concepts:

- Distributed Architecture → The system uses a client-server distributed architecture where multiple servers cooperate to serve client requests.
- Coordinator-based Model → SERVER1 acts as a coordinator node responsible for managing communication between client and replicas.
- Replication → Both SERVER1 and SERVER2 maintain replicated copies of files. Replication improves availability but may introduce temporary inconsistencies.
- Failure Model → The system assumes crash failures where a server may stop responding. Failure detection is performed using timeouts.

3.2 Technical Implementation: Three independent processes are implemented:

Node	Port
SERVER1	5001
SERVER2	5002
CLIENT	dynamic

3.3 The directory structure of the project:

Main folder → distributed_project/

server1.py

server2.py

client.py

Folder 1 → server1/storage/

Folder 2 → server2/storage/

Folder 3 → client/

Each server maintains its own independent file storage directory.

4. Phase 2 – Protocol Design

4.1 Objective: To design a communication protocol for message exchange between the distributed nodes.

4.2 Theory and Concepts

- Message Passing → Distributed systems communicate using message passing rather than shared memory.
- Request–Reply Model → Communication follows a request–reply interaction pattern, where a node sends a request and waits for a response.
- Data Serialization → Messages are encoded using JSON, which provides structured data exchange between nodes.
- TCP Communication → The system uses TCP sockets to provide reliable message delivery.

4.3 Technical Implementation

Message Types → The system uses several message types:

Message Type	Purpose
FILE_REQUEST	Client requests a file
FILE_METADATA_REQUEST	SERVER1 asks SERVER2 about file status
FILE_METADATA_RESPONSE	SERVER2 returns metadata
FILE_CONTENT	File transfer message
ERROR	Error notification

Example request message:

```
{
"type": "FILE_REQUEST",
"pathname": "test.txt"
}
```

Example metadata response:

```
{
"type": "FILE_METADATA_RESPONSE",
"exists": true,
"hash": "abc123",
```

```
"size": 1024

}
```

4.4 Message Framing → To ensure correct transmission over TCP streams, messages are sent with a length-prefixed framing structure.

```
data = json.dumps(message).encode()

length = struct.pack('!I', len(data))

sock.sendall(length + data)
```

---

5. Phase 3 – SERVER2 Implementation

5.1 Objective → To implement SERVER2, which acts as the replica file server.

5.2 Theory and Concepts

- Replica Servers → Distributed systems often maintain replica servers to improve availability and fault tolerance.
- Metadata-based Synchronization → Instead of sending entire files, servers exchange metadata such as hash values to verify consistency.

5.3 Technical Implementation

- SERVER2 performs the following tasks:
- Listens for connections on port 5002
- Receives metadata requests
- Checks whether the requested file exists
- Computes SHA-256 hash of the file
- Sends metadata response

Example implementation:

```
if os.path.exists(file_path):

    file_hash = compute_hash(file_path)

    response = {

        "type": "FILE_METADATA_RESPONSE",

        "exists": True,

        "hash": file_hash,

        "size": file_size

    }
```

SERVER2 does not communicate with the client directly.

---

6. Phase 4 – SERVER1 Implementation

6.1 Objective → To implement SERVER1, the coordinator responsible for processing client requests and validating replica consistency.

6.2 Theory and Concepts

- Coordinator Node → In many distributed systems, one node coordinates communication and decision-making.
- Replica Validation → Before returning data, SERVER1 verifies that replica files are identical.
- Failure Detection → SERVER1 detects failures in SERVER2 using timeout-based detection.

6.3 Technical Implementation

SERVER1 performs the following tasks:

- Accepts client requests
- Checks local storage for the requested file
- Queries SERVER2 for metadata
- Compares file hashes
- Applies decision logic
- Sends the appropriate response to the client

Example hash comparison:

```
if s1_hash == s2_hash:

    send_file(conn, pathname, local_path)

else:

    send_conflict(conn, pathname, local_path)
```

SERVER1 listens on port 5001.

---

7. Phase 5 – Client Implementation

7.1 Objective → To implement the CLIENT application that allows users to request files from the distributed system.

7.2 Theory and Concepts

- Client-Server Interaction → Clients initiate communication and servers respond with requested resources.
- Data Transfer → Files are transferred over the network using Base64 encoding to safely transmit binary data through JSON messages.

7.3 Technical Implementation

The client performs the following steps:

- Connects to SERVER1
- Sends a FILE_REQUEST message

- Waits for response
- Processes response based on type
- Saves received file locally

Example request:

```
request = {

"type": "FILE_REQUEST",

"pathname": filename

}
```

Example file saving:

```
data = base64.b64decode(encoded_data)

with open(filepath, "wb") as f:

    f.write(data)
```

---

8. Phase 6 – Hash-based Consistency Validation

8.1 Objective → To implement efficient replica consistency validation using hashing techniques.

8.2 Theory and Concepts

- Content Hashing → A SHA-256 cryptographic hash uniquely identifies file content. If two files produce the same hash, they are guaranteed to be identical.
- Metadata Comparison → Instead of transferring full files, only hash values are compared, which significantly reduces network overhead.
- Eventual Consistency → Replica divergence is detected when hash values differ.

8.3 Technical Implementation: The following function computes the SHA-256 hash:

```
import hashlib

def compute_hash(filepath):

    sha256 = hashlib.sha256()

    with open(filepath, 'rb') as f:

        while chunk := f.read(4096):

            sha256.update(chunk)

    return sha256.hexdigest()
```

SERVER1 compares the hash values received from SERVER2 with its local hash.

---

9. Failure Detection

The system implements basic failure detection through timeouts.

SERVER1 sets a timeout when connecting to SERVER2:

```
sock.settimeout(5)
```

If SERVER2 does not respond within the timeout period, SERVER1 assumes it is unavailable and continues processing using its local replica.

---

10. System Workflow

- CLIENT sends a file request to SERVER1.
- SERVER1 checks its local storage.
- SERVER1 requests metadata from SERVER2.
- SERVER2 returns file metadata including hash value.
- SERVER1 compares hash values.
- Based on comparison:
    - identical replicas → send file
    - different replicas → report mismatch
    - file only on one server → return available file
    - file missing → send error message

---

11. Conclusion

This project successfully demonstrates the implementation of a basic distributed file retrieval system with replica validation.

The system showcases several important distributed computing concepts, including:

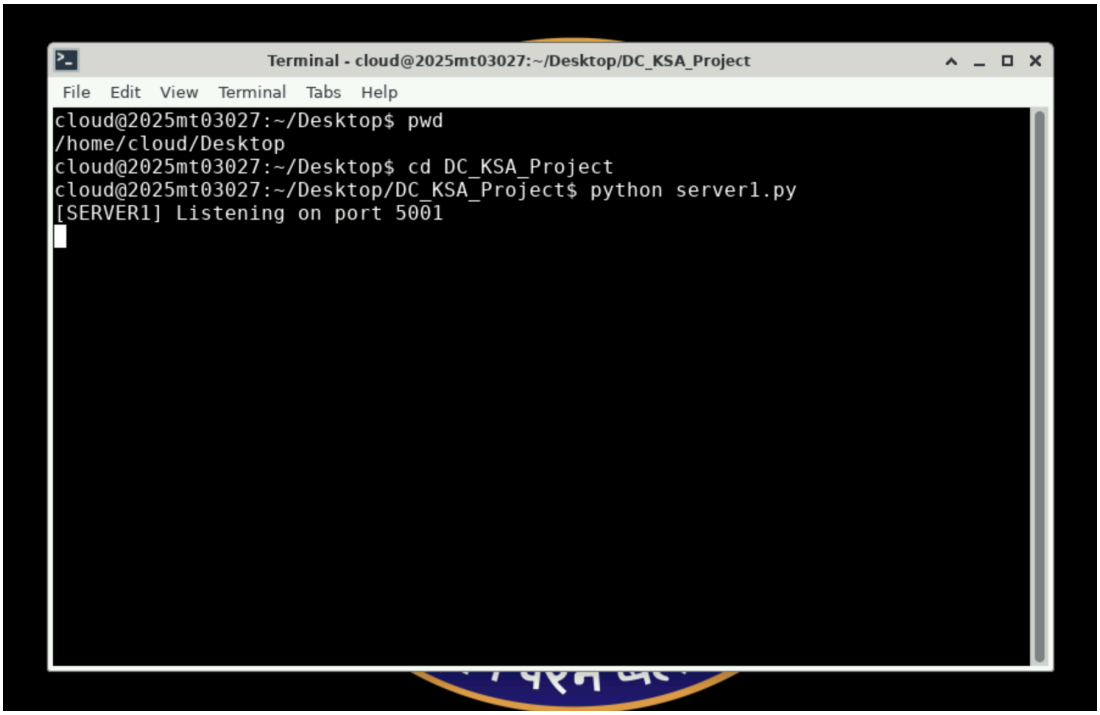
- client-server architecture
- replica-based storage
- message passing communication
- hash-based consistency verification
- failure detection through timeouts

By combining these concepts, the system ensures that the client receives correct file data while also detecting potential inconsistencies between replica servers.

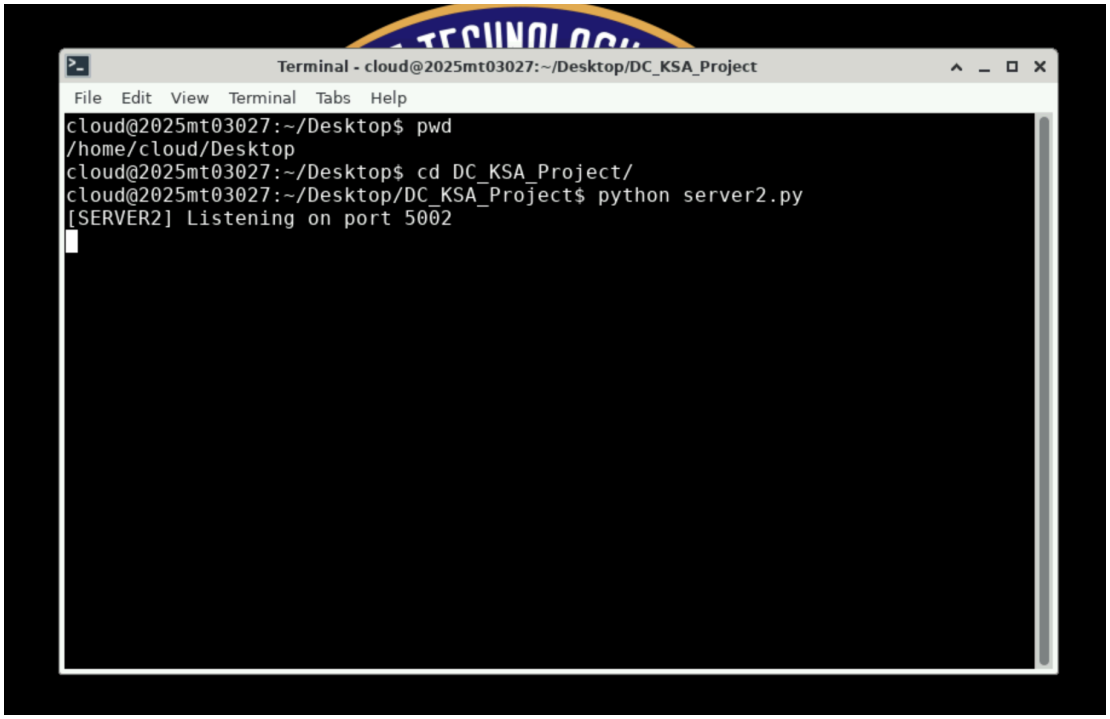
Refer below for Screenshots:

Working Screenshots

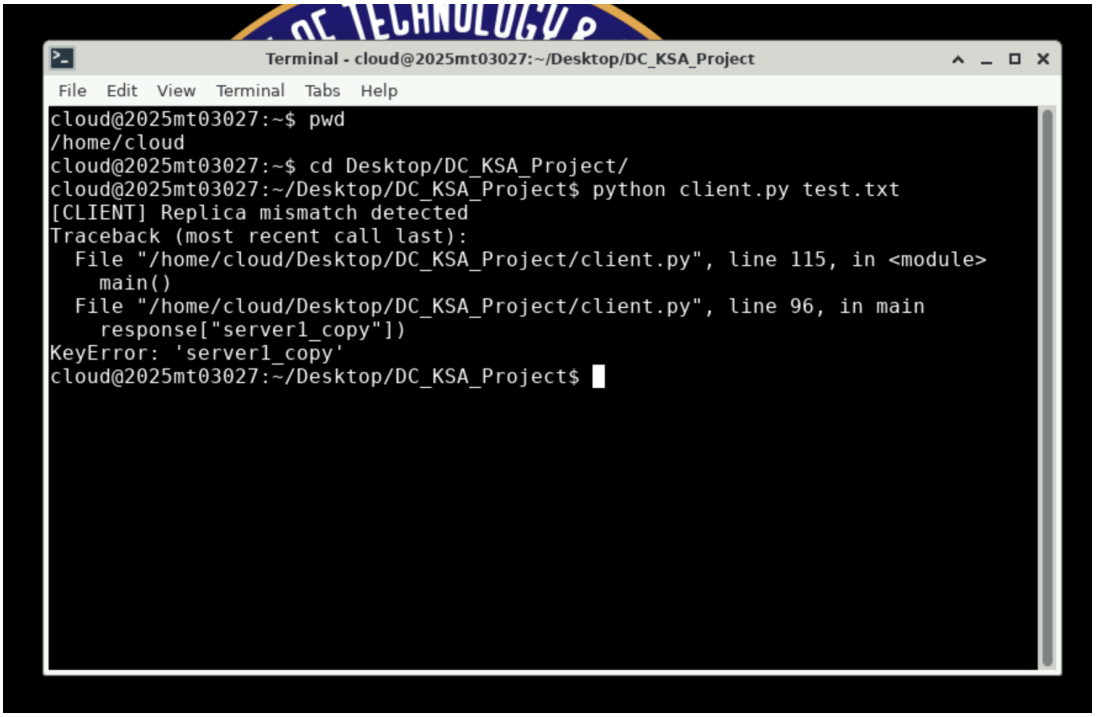
1. Connecting Server1

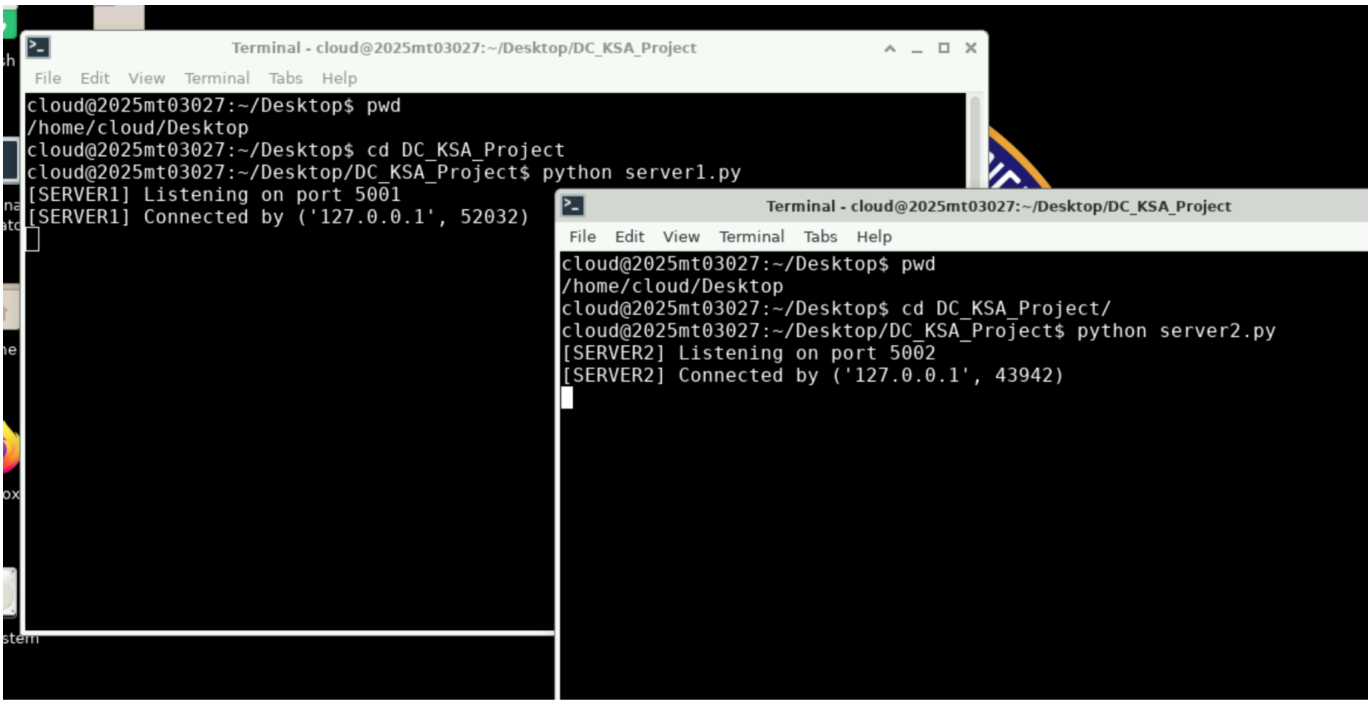


2. Connecting Server2

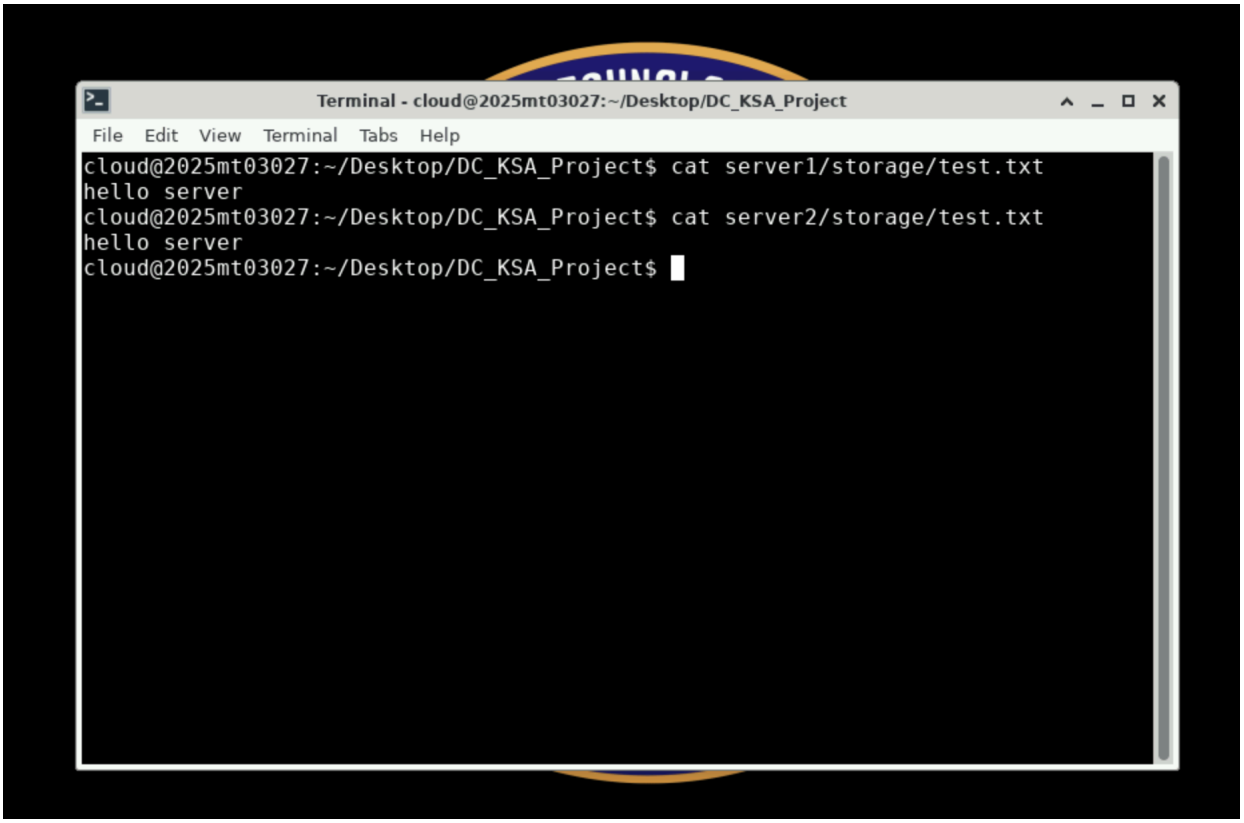


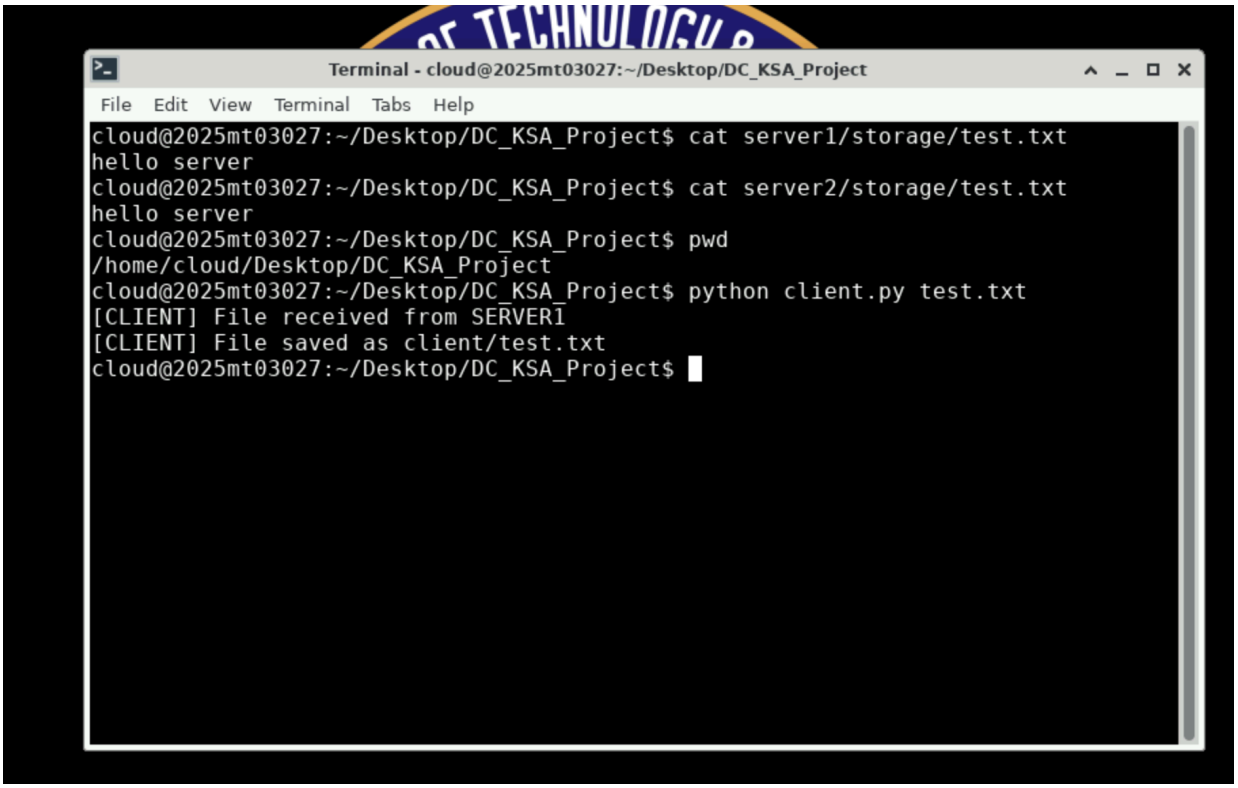
3. When File content on both servers are different



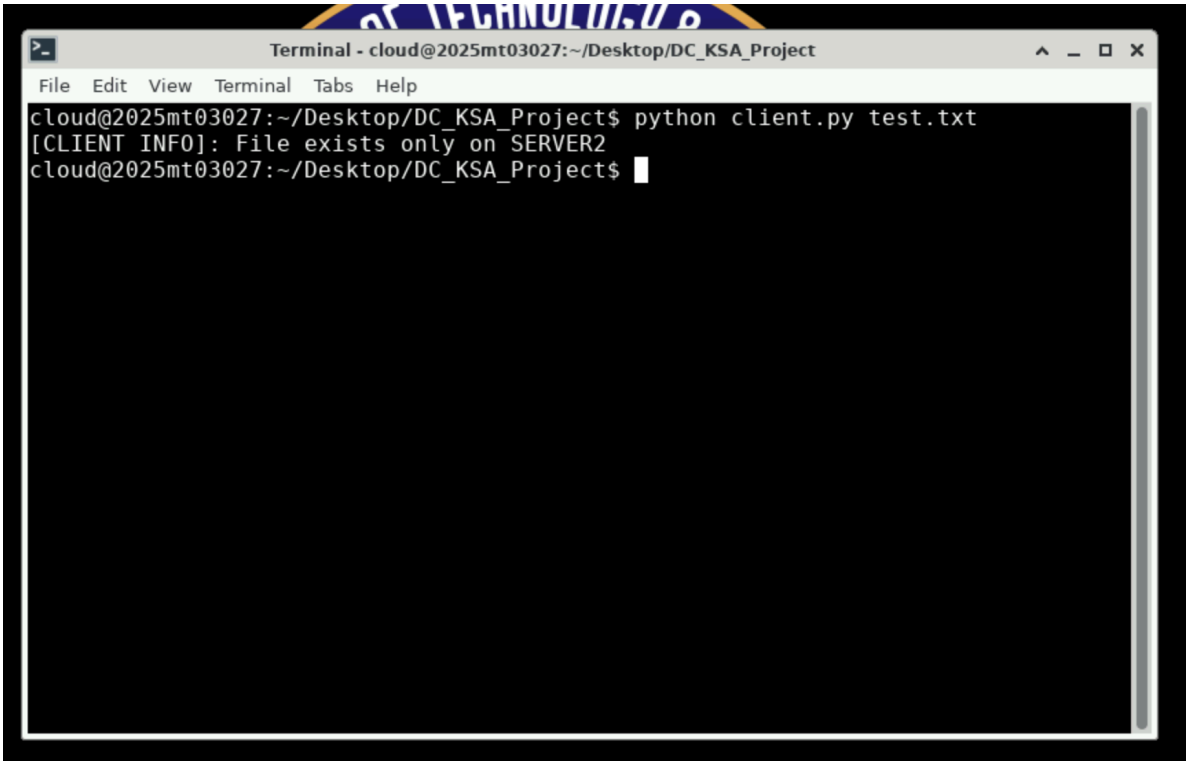


4. File content same on server1 and server2

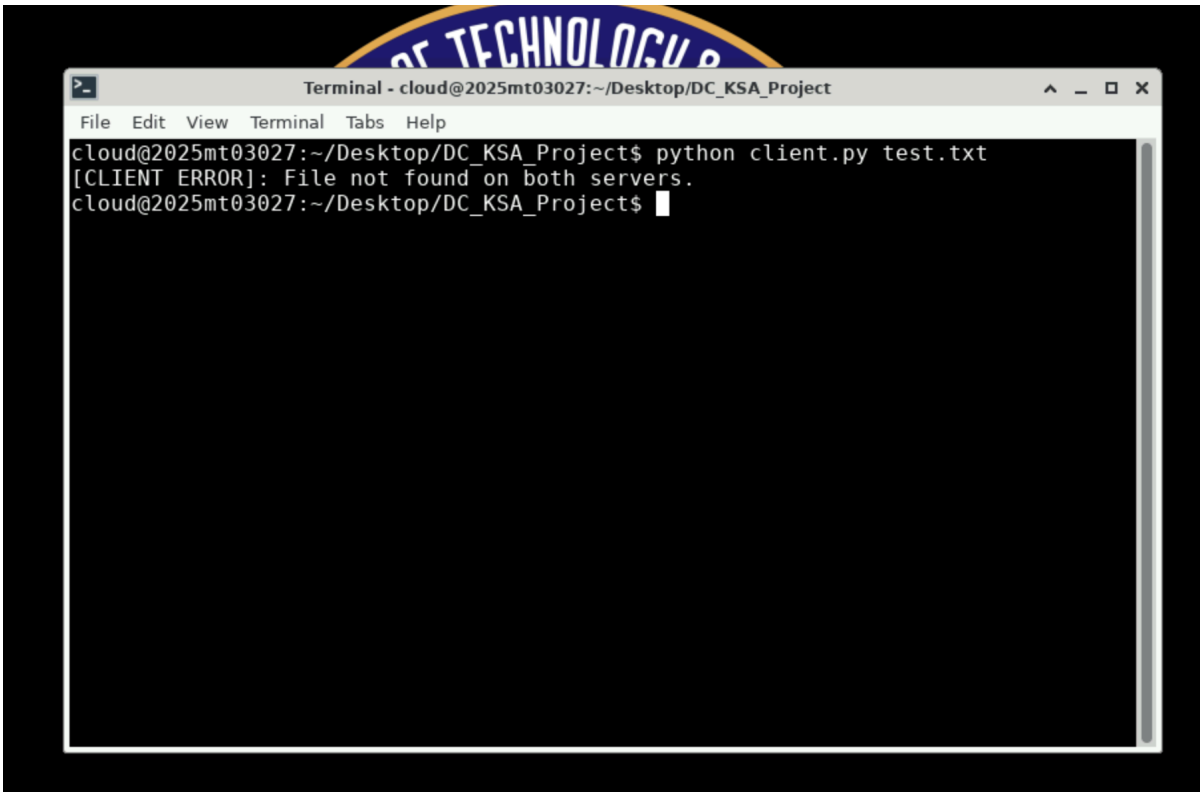
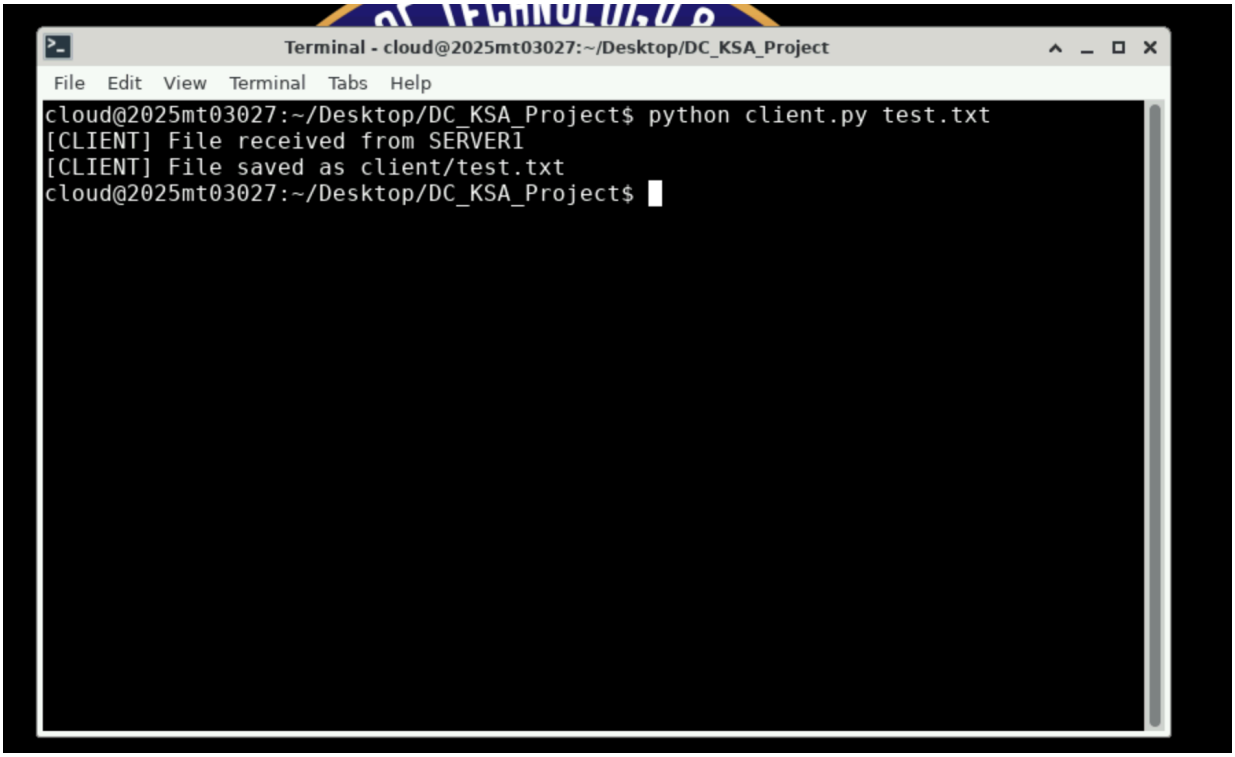




5. File present ONLY on Server2



6. File not present on BOTH Servers



7. File present ONLY on SERVER1